

OSDU SPI Stack

A Complete Guide to the Azure-Native OSDU Deployment System

Source material compiled from the [Azure/osdu-spi-stack](#) repository

September 2026

Contents

About This Document

Part I: The Problem

- What OSDU Is
- The Provider Problem
- What SPI Stack Actually Is
- What SPI Stack Is Not

Part II: The Shape of the System

- Four Owners, Four Boundaries
- Where Things Run
- Profiles: Three Sizes of Truth
- The Services and How They Depend on Each Other
- Data: What Went to Azure and What Stayed
- An Asynchronous Path That Does Not Work

Part III: The Foundations

- The Azure-Only Bet
- The Three That Stayed
- Letting Azure Run the Cluster
- Compliance by Construction: The Local Helm Chart
- Capacity as Application Code
- Availability Zones Are a Discovery Problem
- Bicep, and Why the CLI Stays Thin

Part IV: GitOps and the Deliberate Pause

- What GitOps Means Here
- Ordering Is the Design
- The Deliberate Pause
- Where the GitOps Objects Live
- One Owner per Object
- When Reconciliation Gets Stuck

Part V: Identity, Which Is Two Different Problems

- Outbound: Secrets That Do Not Exist
- Inbound: Rewriting Identity at the Door
- Compliance by Construction

Three Planes of Secrets

Moving a Certificate Authority Across a Namespace

Part VI: Getting In

Three Products, Not One with Switches

Binding to Someone Else's Gateway

A Certificate in the Wrong Namespace

Port 80 Is Always Open

Six Places an Endpoint Can Fail

Two Documented Rough Edges

Part VII: Making an Empty OSDU Useful

The Three Things That Must Exist

Bootstrap as Data

Loading 1,386 Schemas

The Image Lock

Part VIII: The Shared Environment

The Setup

Deliberately Slower Than a Branch

Four Lifetimes, Four Verbs

Failing Closed

Readiness as an API

A Deploy Is a Lock File Edit

Trusting a Repository Without Trusting Its Pull Requests

Trusted and Canonical Are Different Questions

Why the Client ID Must Survive

Part IX: Fifty Minutes, Start to Finish

Before Anything Is Created

Azure

The Cluster

The Pause

Meanwhile, Flux Is Still Working

Five Milestones, Not One

Coming Back Down

Part X: The Things That Are Not True

Part XI: The Register

Glossary

Appendix: The Decision Register

Sources

About This Document

This is a single, self-contained explanation of the **OSDU SPI Stack**: what it is, what problem it solves, how it is built, and the reasoning behind the choices its authors made.

It is written for a technically literate reader who is not assumed to already know OSDU, Kubernetes, GitOps, or Azure's identity model. Each of those is introduced where it first matters.

The material is drawn from the project's own documentation: its architecture overview, nine subsystem design guides, and a register of thirty-four architectural decision records. The decision records are unusually valuable because they preserve not just what was chosen but what was rejected, and why. Where this document describes a trade-off, the trade-off is the project's own stated reasoning, not an outside assessment.

A note on honesty: this project documents its own gaps with unusual candor. It records where an intended behavior is not yet implemented, where a component does not actually work, and where a convenient assumption is false. Those admissions are preserved here rather than smoothed over, because they are often the most interesting parts of the story.

Part I: The Problem

What OSDU Is

The energy industry has a data problem that is fundamentally a translation problem.

A single oil and gas operator might hold seismic surveys, well logs, drilling records, production measurements, and geological interpretations, gathered over decades, stored in dozens of incompatible systems, produced by competing vendors who each had their own ideas about what a “well” is and how to describe one. Merging two companies means merging two of these archives. Bringing in a new analysis tool means writing yet another adapter. The data is enormously valuable and almost impossible to move.

OSDU is the industry’s collective answer. It is an open standard and an open-source reference implementation, developed under the OSDU Forum, that defines a common data platform for subsurface and energy data. Rather than another proprietary archive, it specifies a set of services with agreed-upon APIs: a **storage** service that holds records, a **schema** service that defines what those records may look like, a **search** and **indexer** pair that make them findable, a **legal** service that tracks the compliance tags governing who may see what, an **entitlements** service that answers authorization questions, a **partition** service that keeps tenants separate, and others for files, workflows, and unit and coordinate conversions.

The promise is that if your data is in OSDU, any OSDU-compatible tool can work with it, and you are not captive to the vendor who put it there.

The Provider Problem

OSDU was designed to run anywhere. That ambition creates its own difficulty.

Every one of those services needs somewhere to actually store things. Records need a document database. The entitlements service needs a graph database, because group membership is naturally a graph. Files need object storage. Search needs a search engine. Services need to tell each other when something changed, which needs a message bus. And all of it needs secrets management and an identity provider.

OSDU’s answer is a provider model. The core business logic of each service is shared, and the storage and messaging specifics are pushed behind swappable provider implementations. In the OSDU source tree this shows up as directories like `storage-core` alongside `storage-azure`, `storage-aws`, `storage-gc`, and so on: one set of rules, several sets of plumbing.

This is a **service provider interface**, and it is the seam that gives this project its name. SPI Stack is concerned with exactly one side of that seam: the Azure one. It reads only the `-core` and `-azure` parts of OSDU and deliberately ignores the rest.

What SPI Stack Actually Is

Here is the gap SPI Stack fills.

OSDU’s Azure provider code knows how to talk to Azure Cosmos DB, Azure Service Bus, Azure Storage, Azure Key Vault, and Microsoft Entra ID. It does not know how to *create* any of them. Getting from an empty Azure subscription to a working OSDU installation means provisioning roughly fifty Azure resources in the right order with the right permissions, standing up a Kubernetes cluster, deploying and wiring together a dozen

interdependent services and several pieces of middleware, and then performing a series of one-time bootstrap operations without which the system technically runs but cannot accept a single record.

Done by hand, this is a multi-day expert exercise, and it is not reproducible.

SPI Stack compresses it to one command:

```
spi up --env dev1
```

Forty-five to fifty minutes later, mostly spent waiting on Azure to build a Kubernetes cluster, there is a running OSDU environment.

The project has three moving parts:

- **A Python command-line tool** called `spi`, which orchestrates everything.
- **A set of Bicep templates**, Azure's declarative infrastructure language, which describe the Azure resources.
- **A tree of Kubernetes manifests and Helm charts** in the same Git repository, which describe the in-cluster workloads and are applied continuously by a GitOps engine called Flux CD.

What SPI Stack Is Not

The project states this plainly and repeats it, so it is worth stating plainly here: **this is not a production configuration**.

That is a deliberate scope decision, not an unfinished feature. Several specific things follow from it:

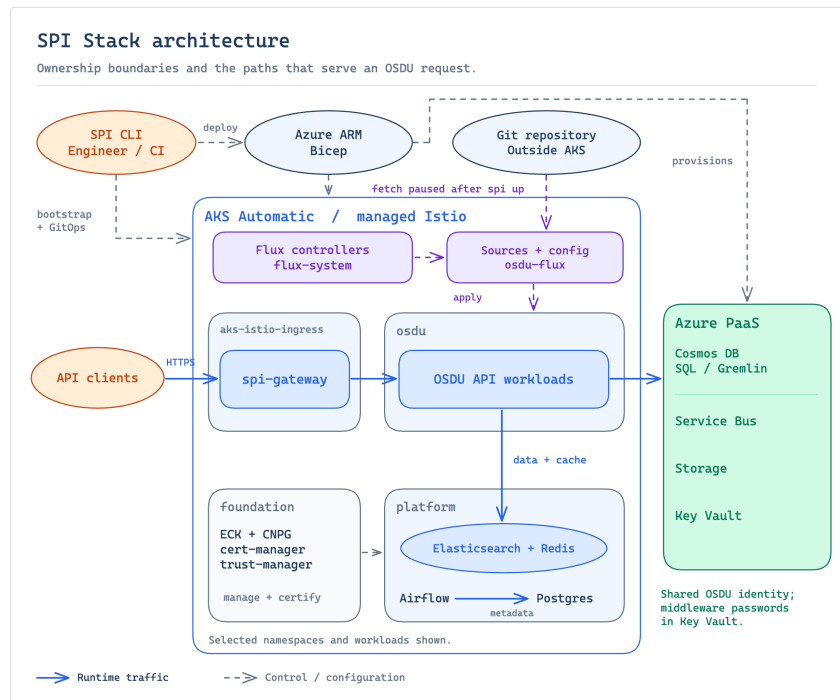
- All OSDU services share a single Azure managed identity. One service's Azure permissions are not isolated from another's.
- There is no backup or disaster recovery for the in-cluster databases.
- There is no automated credential rotation.
- Middleware is sized for a working test environment, not for production load or resilience.

The intended users are developers building against OSDU, platform engineers evaluating whether OSDU on Azure is viable, and, importantly, the continuous integration systems of the eight Azure OSDU service forks that use a shared SPI Stack environment as their test target. That last use case drives a surprisingly large amount of the system's design, and it is the subject of Part VIII.

Part II: The Shape of the System

Four Owners, Four Boundaries

The clearest way into SPI Stack is to notice that four different things own four different parts of the running system, and most of the design exists to keep those boundaries clean.



The CLI and Git are outside the cluster; Flux, the ingress gateway, OSDU services, and middleware run inside it. Provisioning and reconciliation are control paths, separate from the API requests served by the workloads.

The CLI and Bicep own Azure. Resource group, Kubernetes cluster, networking, identities, databases, message buses, storage, and the initial configuration seeded into the cluster. This runs when a human or a CI job explicitly runs it, and never otherwise.

Flux owns the Kubernetes workloads. It reads manifests from the Git repository and applies them in dependency order, continuously, without anyone asking.

Kubernetes controllers and operators own the things they manage: scheduling pods, keeping database clusters healthy, issuing TLS certificates, distributing certificate authorities. They keep working regardless of what Flux or the CLI are doing.

The human operator owns the decisions: when to pull new changes from Git, when to refresh service images, when to delete the environment, and how to diagnose it when something is wrong.

The single most useful thing to understand about this split is a negative claim the documentation makes explicitly: **a successful CLI exit is not a readiness check**. The command can finish, report success, and return you to your shell while Flux is still bringing up half the system. This is not a bug. Provisioning and convergence are genuinely separate concerns with separate timelines, and the project refuses to pretend otherwise.

Where Things Run

Kubernetes organizes workloads into namespaces, which are logical partitions inside a cluster. SPI Stack uses three of its own, plus three it does not own.

Namespace	What lives there	Owner
<code>foundation</code>	Operators: ECK, CloudNativePG, cert-manager, trust-manager	SPI
<code>platform</code>	Stateful middleware: Elasticsearch, Redis, PostgreSQL, Airflow, TLS certificates	SPI
<code>osdu</code>	The OSDU services themselves, plus bootstrap jobs	SPI
<code>osdu-flux</code>	SPI-owned GitOps objects and configuration inputs	SPI
<code>flux-system</code>	The Flux controllers	Azure (AKS extension)
<code>aks-istio-system</code> , <code>aks-istio-ingress</code>	Managed service mesh and ingress	Azure (AKS)

The three-way split of SPI's own workloads is deliberate and is worth a moment, because the obvious simplification is to collapse it.

The separation of `foundation` from `platform` puts the operators in a different namespace from the things they operate. An operator in Kubernetes is a controller that manages a complex stateful system on your behalf: ECK manages Elasticsearch clusters, CloudNativePG manages PostgreSQL clusters. The temptation is to put the operator next to what it manages. The project rejected that, so a bad operator upgrade does not share a blast radius with the database it is supposed to be caring for.

The separation of `osdu` from `platform` matters for a more mechanical reason: only `osdu` gets Istio service mesh sidecars injected. Every OSDU service pod runs alongside a proxy that intercepts its traffic. The middleware does not, which turns out to matter enormously for reasons covered in Part V.

Profiles: Three Sizes of Truth

Not every use of the stack needs all of it. SPI Stack offers three profiles, selected at deployment time.

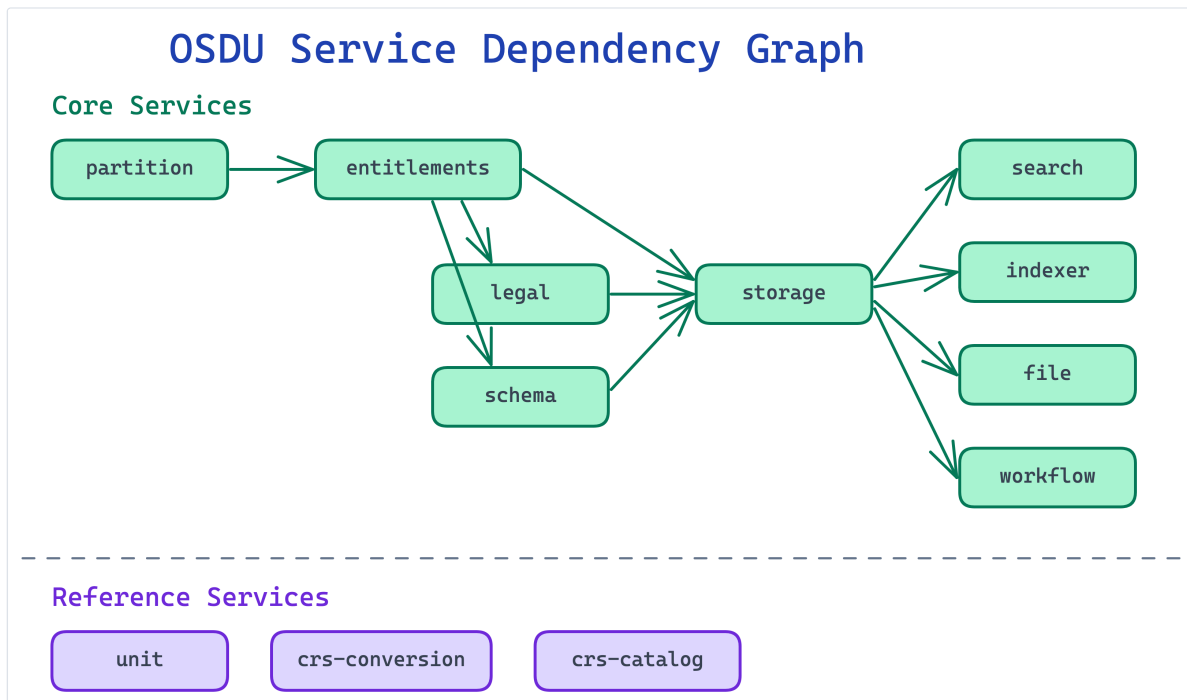
Profile	What Flux deploys	Who wants it
<code>bare</code>	Nothing. Empty workload trees, infrastructure only	Someone iterating on Azure infrastructure or identity
<code>minimal</code>	Operators and middleware, no OSDU services	Someone working on the platform layer itself
<code>core</code>	Everything: middleware, OSDU services, bootstrap jobs, schema loading	Someone evaluating OSDU

There is a catch that the documentation is careful to flag: **profiles select Kubernetes workloads, not a smaller Azure bill**. All three profiles provision the full Azure estate, including the Kubernetes cluster and all the PaaS services. A `bare` deployment is not a cheap deployment. It simply has nothing running on top.

`minimal` has an interesting origin story. The profile enum originally advertised a value called `full` that had no implementation behind it at all. A user could pass `--profile full`, watch Azure resources provision for about forty-five minutes, and only then hit a failure when Flux tried to reconcile a directory path that did not exist. The fix was not to build `full`. It was to delete it and add `minimal`, a profile that reproduces the middleware layers exactly and stops precisely where the OSDU services would begin, and then to add unit tests asserting that every path a profile can reference actually exists and that no dependency points at an undeclared target. The failure mode moved from a forty-five-minute cloud timeout to a test that fails in milliseconds.

The Services and How They Depend on Each Other

The `core` profile deploys ten core OSDU services: partition, entitlements, legal, schema, storage, search, indexer, indexer-queue, file, and workflow. Three reference services follow: unit, crs-conversion, and crs-catalog.



Partition and entitlements sit upstream of legal, schema, and storage; search, indexer, file, and workflow depend on storage; the reference services stand alone.

The shape is roughly a funnel. **Partition** is upstream of nearly everything, because it is how a service discovers which database and which message bus belong to the tenant making the request. **Entitlements** is upstream of nearly everything else, because it answers the authorization question. Legal, schema, and storage build on those. Search, indexer, file, and workflow build on storage.

One important clarification the project makes: this is the *runtime call* dependency graph, which is a different thing from the *deployment order*. Flux has its own ordering, described in Part IV, and the two graphs are not the same.

Data: What Went to Azure and What Stayed

A client request arrives at the cluster's gateway, gets routed to an OSDU service, and that service needs to read or write something. Where does it go?

Mostly to Azure:

- **Cosmos DB (SQL API)** holds records and metadata, one account per partition.
- **Cosmos DB (Gremlin API)** holds the entitlements graph, shared across the environment.
- **Azure Storage** holds file blobs and tables.
- **Azure Service Bus** carries events between services, one namespace per partition.
- **Azure Key Vault** holds configuration values and credentials.

But three things stayed inside the cluster, and each stayed for a different reason, which Part III examines.

An Asynchronous Path That Does Not Work

Here is the first of this project's candid admissions, and it is a good one, because it illustrates how a principled architectural decision can have a concrete, visible cost.

When a record changes in OSDU, the change is supposed to propagate to the search index asynchronously. Storage emits an event to Service Bus. A service called **indexer-queue** consumes that event and invokes **indexer**, which writes to Elasticsearch. Search then finds it.

On a default SPI Stack deployment with the community-built container images, that path does not run.

The reason is a direct consequence of a security decision. SPI Stack disables key-based and shared-access-signature authentication on every Cosmos DB account and Service Bus namespace, on purpose, so that no usable credential exists anywhere in the deployment. The only way in is Microsoft Entra ID, through a mechanism called Workload Identity. The community `indexer-queue` image, however, builds a Service Bus connection string at startup regardless of whether Workload Identity is configured. It reads the placeholder value that SPI Stack writes, which is literally the string `DISABLED`, and cannot authenticate.

The infrastructure provides no fallback, by design. Making this path work requires a replacement image that speaks Workload Identity. The project documents the gap rather than quietly re-enabling keys to hide it, and this tension between “compliant by construction” and “works with the images that exist today” recurs throughout the system.

Part III: The Foundations

The Azure-Only Bet

The first decision in the register is also the one everything else rests on.

OSDU services need document storage, graph storage, object storage, asynchronous messaging, secret management, and an identity provider. There are two coherent ways to supply them. The **portable** approach runs everything inside Kubernetes: PostgreSQL for documents, MinIO for objects, RabbitMQ for messages, Keycloak for identity, Kubernetes Secrets for secrets. Deploy it anywhere. The **managed** approach uses the cloud provider's own services.

SPI Stack chose managed, completely, and accepted the consequence: **the stack is Azure-only by design, not by default**. Running it elsewhere is a fork, not a flag.

The reasoning is worth following, because it inverts an instinct. "Cloud native" is often heard as "runs in Kubernetes." Here the cloud-native choice was to push state *out* of Kubernetes. Two arguments drove it.

The first is that the portable approach is not free. Running five stateful systems in-cluster means owning their backup, their upgrades, their monitoring, and their capacity planning. For a dev and test platform, that is a large standing tax paid in exchange for portability nobody in scope actually needs.

The second is decisive: the OSDU Azure provider modules **already target these Azure services**. Cosmos DB, Service Bus, Storage, Key Vault, and Entra ID are what `storage-azure` and its siblings are written against. Choosing the portable path would not have been configuration. It would have meant not using the Azure provider at all, which is the entire point of the project.

The consequences ripple outward. Five stateful systems leave the cluster, so the cluster needs fewer and smaller nodes. There is no in-cluster backup story for them because they are not in the cluster. Workload Identity becomes mandatory rather than optional, because there is now no other way to reach the data. And the CLI must provision all of it *before* Flux can usefully reconcile anything that depends on it, which is why provisioning and reconciliation are sequenced the way they are.

The Three That Stayed

Having moved everything with a managed equivalent to Azure, three stateful systems remain inside the cluster. Each stayed for a different reason, and the differences are instructive.

Elasticsearch stayed because there is no substitute. OSDU's search and indexer services call the Elasticsearch REST API directly. Azure AI Search is a fine product and is not API-compatible with Elasticsearch. Swapping it in would mean rewriting OSDU service code, which is out of scope. This is a hard constraint, not a preference. It runs under ECK, the Elastic Cloud on Kubernetes operator, sized at three nodes with 128 GiB of storage each.

Redis stayed because of a cost and latency judgment. Azure Cache for Redis exists and would work, but OSDU's usage needs custom certificate-authority TLS and specific database isolation, and a managed cache adds network latency and monthly cost to a dev environment that does not need either. It runs from the Bitnami Helm chart with certificates from cert-manager, one master and two replicas at 8 GiB each.

PostgreSQL stayed for the narrowest reason of all: it exists solely as the metadata database for Apache Airflow, OSDU's workflow engine. An externally managed PostgreSQL for a single embedded component's bookkeeping is poor value. It runs under CloudNativePG, three instances, 10 GiB plus 4 GiB of write-ahead log.

The honest total is that the cluster still carries about 450 GiB of Premium SSD for systems the architecture nominally moved to Azure. And because these three issue their own self-signed certificates, their certificate authorities have to be distributed to the OSDU services that talk to them, a problem Part V returns to.

Letting Azure Run the Cluster

Kubernetes is famously configurable, and configuring it is famously a job. Standard Azure Kubernetes Service makes the operator responsible for node pools, service mesh, admission policy, monitoring, storage drivers, and autoscaling. That is a great deal of platform to maintain in order to get to the point of running an application.

AKS Automatic is Azure's answer: a cluster SKU where Microsoft picks the defaults and manages those components. SPI Stack uses it, and the trade is explicit. You stop maintaining the platform layer. In exchange, Azure decides the cluster's shape, and you live with the decisions.

Three of those decisions bite.

The Kubernetes version floor is 1.36, and it is not arbitrary. Below that version, AKS Automatic blocks the creation of mutating admission webhooks. cert-manager, CloudNativePG, and ECK all depend on mutating webhooks. Below 1.36, the middleware layer simply does not work. This also means the stack cannot be deployed in any Azure region that does not yet offer 1.36.

Deployment Safeguards are cluster-wide and cannot be bypassed. AKS Automatic enforces an admission policy requiring every pod to run as non-root, drop all Linux capabilities, disallow privilege escalation, set a seccomp profile, declare both resource requests and limits, and define liveness and readiness probes. Every pod. Including ones from upstream Helm charts written by people who never heard of this policy. This constraint produced one of the more elegant decisions in the register, discussed below.

Some things still need an imperative nudge. Istio is managed, but enabling CNI chaining for it requires a post-deployment `az aks mesh enable-istio-cni` call, because the Azure resource provider rejects the relevant setting at cluster creation time. The declarative path has a hole in it, and the CLI patches the hole.

Compliance by Construction: The Local Helm Chart

The Deployment Safeguards problem has an obvious solution and a better one.

The obvious solution is to take the upstream OSDU community Helm charts and patch them at deployment time, either with a Kustomize post-renderer or with Flux's own post-render hooks, injecting the required security fields as the manifests are rendered. This works. It is also fragile in a specific and nasty way: when the upstream chart changes, the patch can silently stop matching, and compliance quietly disappears until a deployment fails at admission or, worse, until a security review notices.

The better solution, and the one taken, is to stop consuming upstream charts entirely. SPI Stack ships **one local Helm chart**, `osdu-spi-service`, and every OSDU service is deployed through it. The chart bakes the security context, the probes, the resource settings, and the topology spread constraints directly into its templates. A per-service configuration file overrides only the image, the environment variables, and the resource sizes.

The insight generalizes: **the safest place to enforce a policy is at authoring time, inside the artifact, where it cannot drift, rather than at deployment time, where a patch has to keep matching a moving target.**

The cost is real. One chart now serves every OSDU service, so any change to it is a cross-cutting change affecting all of them. Upstream chart versions stop mattering, which means the project tracks image tags instead, and that is why the image lock in Part VII exists.

Capacity as Application Code

AKS Automatic uses Karpenter, a node provisioner that creates virtual machines on demand to fit pending pods, rather than pre-declared fixed node pools. The node shapes are expressed as Kubernetes custom resources.

The question was where those resources should live: in the Bicep templates alongside the cluster, applied imperatively by the CLI, or as ordinary workload manifests reconciled by Flux.

SPI Stack chose workload manifests, on the argument that **the right node shape depends on the workload mix, not on the cluster**. Node shapes should evolve with the applications, through the same pull-request-and-review path as the applications, not through a separate infrastructure change.

Two pools are defined, and their asymmetry is the interesting part.

The **platform** pool, for middleware, uses a fixed machine shape: 8 vCPU, more than 30 GiB of RAM, premium-storage capable, **on-demand only**, and it holds onto nodes until they are completely empty. The **osdu** pool, for stateless services, accepts a range of sizes from 4 to 16 vCPU, runs on **spot instances** with on-demand fallback, and consolidates aggressively when underutilized.

Spot capacity was explicitly rejected for the platform pool. Spot instances are far cheaper and can be reclaimed by Azure with two minutes' notice. For a stateless HTTP service, that is a fine bargain. For a three-node Elasticsearch cluster maintaining a quorum, it is not. The stateful members also carry *required* rather than *preferred* anti-affinity, meaning Kubernetes must place them on separate hosts and will refuse to place them at all otherwise. That is why the platform pool needs at least three machines to function.

Pods are steered to pools using a label the stack owns, `spi-pool`, rather than a built-in Azure label, so that the placement contract belongs to the project rather than to the platform.

Availability Zones Are a Discovery Problem

A small decision with a general lesson.

AKS Automatic requires an explicit list of availability zones for its system node pool. The natural approach is to hardcode the list, since a region's zones are a property of the region.

Except they are not. The usable zones for a given VM size depend on the *subscription*, not just the region. The same machine size can be zonal in one subscription and restricted in another, and Azure capacity shifts over time. Hardcoding produces a failure that arrives late, after the resource group has already been created, as an `AvailabilityZoneNotSupported` error from the ARM deployment engine.

Shrinking the list to be safe does not work either, because AKS Automatic wants the full usable set.

SPI Stack resolves the zones by querying the target subscription's SKU catalogue **before creating anything at all**, including the resource group. If the subscription cannot use the full published set, deployment stops with a preflight error naming the machine size and the region. It refuses to silently degrade.

The generalizable point: what looks like a static configuration constant is sometimes a per-tenant discovery problem, and finding out before you create resources is worth an extra API call.

Bicep, and Why the CLI Stays Thin

A full deployment creates roughly fifty Azure resources with a real dependency graph among them: identities that must exist before role assignments, a cluster whose OIDC issuer must exist before federated credentials can reference it, per-partition resources repeated for each tenant.

Writing that as imperative CLI calls means writing and maintaining a dependency resolver, an idempotency layer, and a retry strategy, and it means ARM will reject your ordering mistakes at deployment time anyway.

SPI Stack declares essentially everything in **Bicep**, Azure's declarative infrastructure language, and keeps the Python CLI deliberately thin. The stated philosophy: **the CLI is not the infrastructure engine, it is the conductor**. Adding a new Azure resource means adding a Bicep module, not teaching the CLI a new resource type.

There are three entry-point templates, and their order encodes a real dependency:

1. `infra/aks.bicep` builds the cluster, its virtual network, NAT gateway, and managed Istio configuration. Its critical output is the cluster's **OIDC issuer URL**.
2. `infra/main.bicep` builds everything else: identities, Key Vault, container registry, the Gremlin graph, shared storage, and each partition's Cosmos DB, Service Bus, and Storage account. It needs the OIDC issuer from step one in order to create the federated credentials that make Workload Identity work.
3. `infra/flux.bicep` installs the Flux extension and points it at the Git repository. It goes last because it depends on the cluster being ready and on the configuration the CLI seeds into it.

Terraform was considered and rejected, as were Azure Verified Modules and a pure CLI orchestrator.

Why the CLI Cannot Be Purely a Conductor

Six things remain imperative, and the list is a good catalogue of the seams that declarative infrastructure does not cover:

- **Creating the resource group.** Bicep can do this at subscription scope, but this implementation keeps it in the CLI.
- **Recovering a soft-deleted Key Vault.** Azure retains deleted vaults by name. Redeploying an environment with the same name hits the tombstone, and recovery is an imperative operation.
- **Fetching kubeconfig and granting the deployer cluster-admin access.**
- **Enabling Istio CNI chaining,** the hole in the declarative path mentioned earlier.
- **Bootstrapping Kubernetes:** creating namespaces, seeding secrets, applying the Istio identity policies.
- **Writing runtime secrets to Key Vault** after Flux has been activated.

The Preview That Cannot Preview Everything

`spi up --dry-run` runs Azure's `what-if` preview, and the documentation is unusually careful to enumerate what it cannot show you.

It previews only the first two templates, not the Flux one. It does not recover a soft-deleted vault, write secrets, or touch Kubernetes at all. And there is a subtle failure that makes a nice illustration of why previewing infrastructure is genuinely hard: **the AKS preview does not return an OIDC issuer URL**, because no cluster is created. So the second template receives an empty issuer and silently omits every federated credential that would depend on it. The preview is missing an entire branch of the real deployment. DNS zone discovery is skipped too, so a DNS-mode preview omits the ExternalDNS resources.

`--dry-run` is also not a complete no-op: it still creates or updates the resource group and its naming tag.

ARM Is Not a Transaction

One more property worth internalizing, because it shapes the entire recovery model: an Azure Resource Manager deployment that fails partway through does not roll back. Resources already created stay created. A failed deployment does not leave you where you started; it leaves you with a partial environment.

This is why `spi up` is designed to be safely re-runnable, and why `spi up` is explicitly *not* a resume-from-step operation. It does not remember where it failed. It reuses the persisted naming suffix and the existing credential seed, resubmits the infrastructure, and re-applies the bootstrap configuration. The same is true in reverse for teardown, which is likewise non-atomic.

Part IV: GitOps and the Deliberate Pause

What GitOps Means Here

Once Azure exists and the cluster is running, something has to deploy the actual workloads: the operators, the middleware, the OSDU services, the ingress routes.

The CLI could do it. That is the obvious extension of what it already does. The project rejected this, and the reasoning names three specific harms: cluster state would not be reconstructable from Git, upgrades would be coupled to the CLI version, and every change to any workload would become a change to the CLI.

Instead, in-cluster state is reconciled by **Flux CD**. The model, called GitOps, is simple to state: the desired state of the cluster lives in a Git repository, and a controller running inside the cluster continuously compares reality against that repository and corrects the difference. Nobody deploys. Everybody commits.

There is a twist. Flux itself is not installed by the CLI. It is installed as an **AKS extension**, declared in Bicep alongside the Azure resources, so that the GitOps engine is itself managed infrastructure that Azure upgrades. The project describes this as letting Azure own Flux.

A single Flux configuration points at the repository, with two top-level entries beneath it: `stack`, which selects the profile's workload tree, and `ingress`, which selects the ingress mode's tree. Which paths those resolve to is computed in Bicep from the profile and mode, not chosen independently by the operator.

Ordering Is the Design

Kubernetes has hard ordering constraints that it does not enforce for you. Custom resource definitions must exist before custom resources that use them. Operators must be running before the resources they manage appear. cert-manager must be up before anything asks for a certificate. Middleware must be serving before its consumers start.

Apply everything at once and it does eventually converge, through a storm of crash loops and “no such kind” errors that resolve themselves. The problem is not that it fails. The problem is that **a real failure is invisible inside that noise**.

SPI Stack encodes the ordering explicitly. Flux's `dependsOn` mechanism wires the workload groups into a dependency graph, and every group waits for the health of what it depends on before starting. The `core` profile's graph runs roughly:

- Namespaces first, with nothing before them.
- Node pools and the operators (cert-manager, ECK, CloudNativePG) next, in parallel, since none depends on another.
- trust-manager after cert-manager. Elasticsearch after ECK and node pools. Redis after cert-manager and node pools. PostgreSQL after CloudNativePG.
- Airflow after PostgreSQL, on its own branch.
- A bootstrap layer after trust-manager, Elasticsearch, and Redis are all healthy.
- The OSDU services after bootstrap.
- Partition and entitlements initialization after the services.
- Schema loading, legal-tag seeding, and member seeding after initialization.
- Reference services after schema loading.

Two details reward attention. First, **parallelism is preserved**: the three operators come up simultaneously because nothing forces them into sequence. The graph is a graph, not a queue. Second, some branches are deliberately **non-gating**: legal-tag seeding runs and is visible, but nothing waits for it, so a failure there does not hold up the world. Member seeding, by contrast, *is* treated as a blocker, because without the entitlements root groups a fork cannot run a single positive test.

The cost of this design is stated plainly: adding a component means adding a node to the graph and wiring its edges, and because each group waits for health, one slow operator holds up everything behind it.

The Deliberate Pause

Here is the decision most likely to surprise someone who knows GitOps.

Flux's default behavior is continuous reconciliation. Merge to the main branch and within about a minute every connected cluster begins converging on the new commit. For production, that is the entire value proposition.

For a development and test environment, it is a hazard. Consider the actual scenario: an engineer is thirty minutes into debugging an unusual failure, somebody merges an unrelated pull request, and the environment changes underneath them mid-investigation. The bug they were chasing is now unreproducible, and they do not necessarily know why.

So `spi up` ends by **suspending the Git source**.

The environment is pinned to the commit that was current when it was deployed. Flux keeps running, controllers keep reconciling, pods keep being managed, but no new commits are fetched. Moving forward is an explicit act:

```
spi reconcile --resume .
```

The precision here matters, and the documentation insists on it: **suspension means “stop fetching,” not “stop reconciling.”** The cached revision continues to be applied. If someone edits a live ConfigMap by hand, Flux will still put it back. Suspension is not a freeze, and it is not a snapshot. Helm repositories and other controllers keep their own schedules. External chart sources are not pinned by it.

The safe default and the tool's default were not the same thing, and the project changed the default.

Where the GitOps Objects Live

A small decision forced by a policy, with a clarifying result.

The Flux controllers live in a namespace called `flux-system`, which the AKS extension owns. AKS Automatic protects extension-managed system namespaces with an admission policy, which means the deploying identity **cannot write objects there**. But SPI Stack needs to seed configuration into the cluster before Flux starts consuming it: the Git source definition, the workload groupings, secret seeds, and several configuration inputs.

The resolution: create a separate namespace, `osdu-flux`, and put every SPI-owned GitOps object there. The controllers stay in `flux-system` where Azure put them. The things the controllers act on live somewhere writable.

The separation turns out to clarify something that was always true but easy to miss: **the thing that runs GitOps and the things GitOps owns are different categories**. Conflating them in one namespace was a convention, not a requirement.

One cost is disclosed honestly. Flux's multi-tenancy enforcement, which makes controllers impersonate a restricted identity when applying manifests, is turned **off**, because AKS Automatic blocks that impersonation.

The controllers apply manifests as their own exempt identities instead. That is a real reduction in isolation, recorded rather than hidden.

One Owner per Object

This decision starts with a bug that took real effort to diagnose, and lands on a principle.

TLS was not working. The ingress gateway would come up, but its HTTPS listener never became ready. The cause: two separate Flux groups were both rendering the *same* Gateway object, with different desired states. Each would apply its version; the other would apply its version back. They fought, indefinitely, and because both groups reported themselves healthy, nothing looked wrong.

The fix was to establish an invariant: **every Kubernetes object has exactly one Flux inventory owner.**

The deeper insight is in why this holds even when there is no conflict. Two Flux groups could render byte-identical YAML and still be dangerous, because ownership is not about content. Flux prunes: when a group stops rendering an object it previously rendered, it deletes it. If two groups claim the same object and one stops, it takes the object with it, out from under the other one.

The dangerous thing is the claim, not the content.

This has an awkward consequence for changing ownership. You cannot simply move an object from one group to another in a single release, because the old owner might prune it after the new owner applies it. So the handoff is staged: the old owner first becomes an *empty* group configured not to prune and to orphan rather than delete what it releases. Only a later release removes it. This is why an operator inspecting a running cluster will find a vestigial, empty `spi-gateway` group sitting next to the real `spi-gateway-tls` one. It is not dead code. It is a deliberate intermediate state in an ownership transfer.

When Reconciliation Gets Stuck

Two failure modes are worth understanding, because both look like bugs and neither is fixed by waiting.

Retry exhaustion. A Helm release that fails installation retries a set number of times and then gives up, entering a `Stalled` state with the reason `RetriesExceeded`. The controller then stops trying, permanently. The counterintuitive part is that **re-applying the identical manifest does not help**, because the controller only reacts to a change in the resource's generation, and re-applying the same content changes nothing. Everything downstream waits forever on a dependency that is never going to resolve itself.

`spi reconcile` handles this specific case by annotating stalled releases with a timestamp that resets the failure counter and forces one more attempt. It deliberately touches only releases in that state, leaving everything else alone.

Immutable field drift. This one is genuinely subtle. Kubernetes Jobs have immutable fields in their pod template. AKS Automatic's Safeguards include a mutating webhook that *raises a Job's CPU request to 100 millicores* if it asks for less. So a chart requesting less than 100m produces a live Job that no longer matches the manifest that created it, in a field that cannot be changed. The Helm release wedges in `RollbackFailed`.

The escape hatches do not work. Forcing does not work, because Helm's replace strategy and the API server's patch both fail on immutable fields. Deleting the Job by hand does not work, because the chart will render the same wrong template again. And `spi reconcile` does not fix it, because this is not retry exhaustion.

The actual fix is to make the chart request exactly what the webhook would set anyway. SPI Stack's bootstrap charts request precisely 100m so that the rendered manifest and the mutated live object agree. **The chart is**

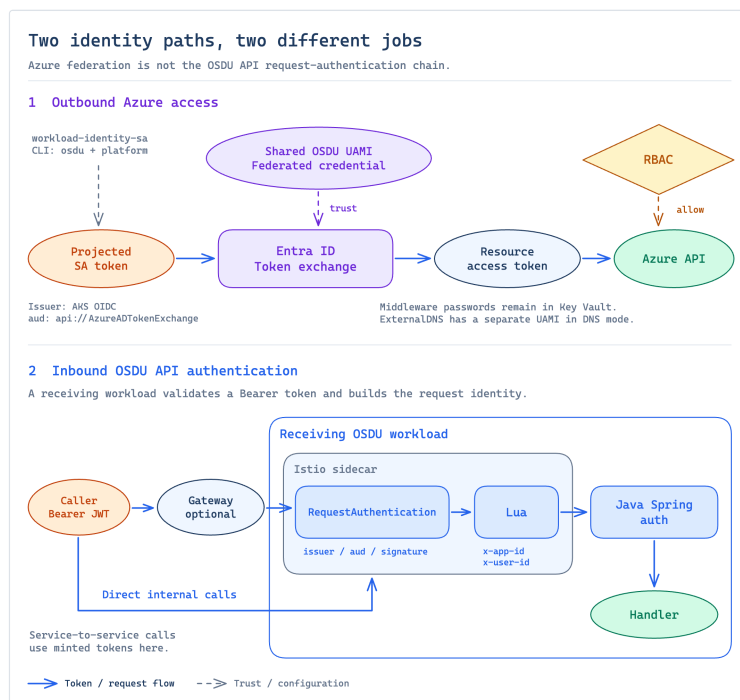
written to match what the platform will do to it, which is an unusual but correct way to resolve a conflict between two systems that both insist on being right.

Part V: Identity, Which Is Two Different Problems

The single most useful framing in this entire system is that **identity here is not one thing**. There are two completely separate mechanisms, with separate failure modes, and the documentation is emphatic that one succeeding tells you nothing about the other.

- **Outbound:** can this pod get a token to talk to Azure?
- **Inbound:** will this API request be accepted by an OSDU service?

A pod can authenticate to Cosmos DB perfectly while every request to its own API returns 401. Conflating the two is the most common way to spend an afternoon debugging the wrong layer.



Federated credentials bind a Kubernetes ServiceAccount to Entra ID; the projected token is exchanged for an Azure access token at runtime.

Outbound: Secrets That Do Not Exist

A pod needs to read from Cosmos DB. Traditionally it holds a credential: a connection string or a client secret, stored in a Kubernetes Secret, mounted into the pod. That credential is a liability. It leaks into logs, into backups, into screenshots. It must be rotated, and rotating it means coordinating everything that holds a copy.

Azure Workload Identity removes it. The mechanism is worth walking through because it is genuinely clever.

Every Kubernetes cluster can act as an OpenID Connect identity provider, issuing signed tokens that assert “this pod is running as this ServiceAccount in this cluster.” AKS Automatic enables this by default. Separately, Entra ID can be told to *trust* a specific external issuer for a specific subject: a **federated credential** saying “tokens from this cluster’s issuer, naming this ServiceAccount, may act as this managed identity.”

At runtime:

1. The CLI creates a ServiceAccount named `workload-identity-sa`, annotated with the managed identity's client ID and tenant ID.
2. A pod uses that ServiceAccount and carries the label `azure.workload.identity/use: "true"`.
3. An admission webhook sees the label, mounts a short-lived signed token as a **file** in the pod, and injects the environment variables pointing at it.
4. The Azure SDK reads that file and presents it to Entra ID, which checks the federated credential and, if issuer, subject, and audience all match, returns a real Azure access token.
5. The pod uses that token against Cosmos DB, which authorizes it by role.

No secret is stored anywhere. The file in the pod is a short-lived assertion of identity, not a credential, and its audience is deliberately distinct from the Azure token it is exchanged for.

The Isolation Trade-Off, Stated Plainly

The natural design gives each service its own managed identity with least privilege: storage service gets Cosmos access, file service gets blob access, nothing gets more than it needs.

SPI Stack rejected this. **All OSDU services share one managed identity**, `spi-stack-<env>-osdu-identity`, holding the union of every permission any of them needs: Key Vault Secrets User, Storage Blob and Table Data Contributor, Service Bus Data Sender and Receiver, container registry pull, and Cosmos data contributor roles on both the SQL and Gremlin accounts.

The gain is operational simplicity: one identity, one set of role assignments, deterministic to provision, no per-service federated credential matrix. The cost is stated without hedging in the project's own words: this **does not isolate one OSDU service's Azure access from another's**. Containment happens at the Kubernetes and service mesh layer instead, not at the Azure layer.

This is exactly the kind of trade that is correct for a dev and test platform and would not be correct for production, which is why the project's scope statement is load-bearing rather than boilerplate.

Two Details That Cause Confusion

Cosmos DB grants are invisible to the usual tooling. Cosmos data-plane access uses Cosmos-native role assignments, not standard Azure role assignments. Running `az role assignment list` will not show them. An engineer verifying permissions with the obvious command will conclude, wrongly, that the grant is missing. Worse, Cosmos role propagation can lag five to fifteen minutes and sometimes requires a pod restart, so a correct configuration can fail for a while and then start working with no intervention.

Container images are pulled by a different identity entirely. The kubelet, not the pod, pulls images, using the cluster's own identity. Workload Identity has nothing to do with it.

Inbound: Rewriting Identity at the Door

Now the other direction, and a genuinely unusual piece of engineering.

The OSDU Azure provider services are Java Spring applications. Their filter chain expects the caller's application identity to arrive in an HTTP **header**, specifically `x-app-id`, not merely inside the bearer token. That filter chain cannot be disabled.

The consequence is that a caller can present a perfectly valid Entra ID token and still be rejected before any business logic runs, with a log line showing an empty `app-id=`. The token is fine. The header is missing.

SPI Stack solves this in the service mesh, using three Istio resources applied to the `osdu` namespace:

- A **RequestAuthentication** that validates incoming JWTs against the configured Entra issuers and audiences, accepting both v1 and v2 token formats.
- An **EnvoyFilter** running a small Lua script that reads the *validated* token's claims and writes `x-app-id` and `x-user-id` onto the request.
- A permissive **PeerAuthentication** governing mesh-internal mTLS.

The security-critical detail is easy to miss and essential: **the filter strips any incoming `x-app-id` and `x-user-id` headers before writing its own**. If it did not, any caller could simply set the header and impersonate anyone. The headers are trustworthy precisely because they are always overwritten from a cryptographically validated token and never passed through.

The result is that the caller's real identity reaches the application. A bootstrap Job arrives as itself, a CI deploy identity as itself, a human as themselves, and the entitlements service decides what each may do.

The disclosed costs: Envoy Lua is now part of the authentication path, so sidecar or key-set drift shows up as an empty `app-id` rather than a clear authentication error; and the script is coupled to specific Entra claim names, so a change in token format requires revisiting it.

A Wrinkle Worth Knowing

A managed identity cannot be the audience of a token. This is an Entra ID property, not a project choice, and it has a practical consequence: on a default environment, an external caller cannot mint a token addressed to the OSDU managed identity. They use the Azure management audience instead. Because the filter projects the caller's *own* application id regardless of what the token was addressed to, the audience does not change who the service sees.

A related utility exists for testing: `spi token` mints an app-only bearer token by creating a Kubernetes ServiceAccount token and exchanging it with Entra ID. It deliberately uses Entra's older v1 endpoint, because the v2 endpoint emits a claim named `azp` and omits `appid`, and the Spring filters read `appid`. The token goes to standard output alone so it can be captured into a variable, with status messages sent to standard error. Three test identities exist: a deploy identity, a member identity for non-admin cases, and a deliberately unauthorized identity for testing that 401s actually happen.

Compliance by Construction

The original infrastructure enabled both Entra authentication and traditional key-based authentication on Cosmos DB and Service Bus, on the assumption that an organizational policy would disable the keys afterward.

The project identified three problems with that. The key material is present in the deployment in the meantime. The security posture depends on a policy that may be absent or merely advisory. And in tenants where policy *denies* creating such resources with local auth enabled, the deployment simply fails.

So local authentication is now hardcoded off. Every Cosmos account and Service Bus namespace is created with key and shared-access-signature authentication **disabled**, unconditionally. Storage shared-key access is disabled too. Workload Identity is not the preferred path; it is the only path.

The project describes this as moving compliance **from “policy later” into “code now.”**

And it has a price, already met in Part II: the community `indexer-queue` image cannot authenticate, because it builds a connection string regardless. The `DISABLED` placeholder secrets that remain in Key Vault exist purely so

that clients expecting those key names find *something*; they are schema compatibility, not fallback credentials. The project declines to weaken the data plane to accommodate an image that has not caught up.

Three Planes of Secrets

Not everything can be a token. SPI Stack sorts secrets into three stores by what kind of thing they are.

Entra ID tokens, obtained at runtime, for all Azure PaaS access. Never stored.

Azure Key Vault, for durable values and metadata: tenant ID, vault URI, per-partition Cosmos endpoints, Service Bus namespaces, storage accounts, and mirrored middleware credentials. Bicep writes the Azure-derived values; the CLI writes the middleware-derived ones after Flux is activated.

Kubernetes Secrets, for in-cluster middleware passwords only. These cannot be tokens because Elasticsearch, Redis, and PostgreSQL authenticate with passwords, full stop.

Plain configuration that is not sensitive at all, such as endpoints and identifiers, lives in an ordinary ConfigMap called `osdu-config`.

The Credential Seed, and Why Deleting It Is Dangerous

Middleware passwords are generated once and preserved in a Kubernetes Secret named `spi-secrets`. On every subsequent run the CLI reads this seed and reuses it; only missing fields are generated fresh.

This makes re-running `spi up` safe. It also creates a failure mode the documentation flags explicitly: **delete the seed and a later run generates new passwords while the existing databases still hold the old ones**. The environment breaks in a way that looks nothing like its cause.

The seed feeds copies into the specific Kubernetes Secrets each component expects, and the charts are configured to consume those CLI-created Secrets rather than generate their own. The Redis chart points at `existingSecret: redis-credentials`; CloudNativePG references CLI-created PostgreSQL secrets; Airflow reads a CLI-written connection string. One component gets special handling: Airflow's Fernet encryption key must be URL-safe base64 of exactly 32 random bytes, not the ordinary alphanumeric password format.

Rotation Does Not Exist, and the Project Says So

There is no supported end-to-end credential rotation. Not "it is manual"; there is no command.

The reasoning is honest about scope. Real rotation would have to coordinate the persistent seed, the database's own authentication state, the Kubernetes copies, the Key Vault copies, and any cached credentials, with a rollback path for partial failure. Half of that is not rotation, it is an outage.

The documentation also pre-empts the obvious wrong intuition: deleting a Secret and letting reconciliation recreate it does **not** rotate anything, because the chart consumes the CLI-owned Secret and the CLI reuses the seed. You get the same password back.

It also notes, drily, that base64-encoded values in a Kubernetes Secret are encoded, not encrypted.

Moving a Certificate Authority Across a Namespace

One more problem, with a tidy solution and a sharp edge.

Elasticsearch and Redis run in `platform` with self-signed TLS certificates. The OSDU services in `osdu` need to trust those certificate authorities, and the Java service charts expect to read the CA material from a Secret in their own namespace so an init container can build a Java truststore.

Kubernetes Secrets do not cross namespaces. There is no built-in mechanism.

The rejected options are instructive: a Job with its own identity to copy them (imperative, needs credentials), a general-purpose replication controller (too broad a permission), or having the CLI copy them on every run (reintroduces an imperative tail the GitOps model was meant to remove).

The chosen answer is **trust-manager**, a controller whose entire job is distributing certificate authorities. Two declarative Bundle resources mirror the Redis and Elasticsearch CAs into `osdu`. When a certificate rotates, the copies follow automatically. The blast radius is deliberately constrained: trust-manager is authorized to write exactly two named Secrets and its default public-CA package is disabled.

The Sharp Edge

Refreshing the CA Secret is **not** end-to-end trust rotation.

The Java truststore is built by an **init container**, which by definition runs once, when the pod starts. Updating the Bundle updates the Secret, and every running pod continues using the truststore it built at startup. Rotating a certificate authority requires recreating the affected pods and then verifying they actually trust the new chain.

This is a specific instance of a general truth worth carrying away: **updating a Kubernetes object is not the same as updating the running application's state.**

One Static Exception

There is also a single odd-looking rule in the mesh: a DestinationRule that disables Istio mutual TLS for traffic to Redis.

The reason is a real incompatibility. Redis clients here use native Redis TLS. Istio, doing its job, wraps outbound connections in its own TLS. The result is TLS inside TLS, which the Java Redis client cannot unwind. Disabling mesh TLS for that one destination fixes it.

The documentation is careful to note what this does and does not mean: **disabling mesh TLS is not disabling encryption.** The application's own Redis TLS is still there. Only the redundant outer layer is removed.

Part VI: Getting In

Three Products, Not One with Switches

Exposing the cluster to clients has three genuinely different audiences.

A developer spinning up a personal environment owns no DNS zone and wants a working HTTPS URL without administrative overhead. A team with a real Azure DNS zone wants proper hostnames, per-service names, and certificates in their own domain. A smoke test or a debugging session wants the shortest path to a response and does not care about TLS at all.

The tempting design is one ingress configuration with flags. SPI Stack instead treats these as **three self-contained products**, each with its own complete tree of manifests, selected by `--ingress-mode`.

Mode	Hostname	TLS	For
<code>azure</code> (default)	Azure-assigned <code><label>.<region>.cloudapp.azure.com</code>	Let's Encrypt	Dev and test with no owned DNS
<code>dns</code>	Environment-prefixed names in your Azure DNS zone	Let's Encrypt	Team environments with owned DNS
<code>ip</code>	Raw gateway IP	None	Isolated debugging only

Adding a fourth mode is a new directory and an enum value, not surgery on a conditional. The cost is some duplication between trees, which the project accepts as the price of each mode being readable on its own.

Binding to Someone Else's Gateway

AKS Automatic's managed Istio creates an ingress workload and a load-balancer Service. It does **not** create a Service per Gateway object. A Gateway that does not say otherwise will look for a Service that nobody is going to create.

The obvious fix is for the stack to deploy its own ingress workload and load balancer. That means a second public IP, a second thing to manage, and paying twice for infrastructure Azure already provisioned.

So SPI Stack binds its Gateway directly to the add-on's existing ingress Service, and stops trying to own it.

This creates an interesting ownership arrangement, because the `azure` mode needs a DNS label applied to that Service, and two separate AKS Automatic protections block the natural routes: the managed namespace policy blocks writing to the Service, and a deny assignment on the node resource group blocks touching the public IP directly.

The resolution is narrow and precise. Flux server-side applies a partial manifest that carries **exactly one annotation** and claims nothing else about the Service. Azure's cloud controller sees the annotation and assigns the DNS name to the public IP. Pruning is explicitly disabled on that manifest, so switching ingress modes or removing the stack never deletes the Service that Azure owns.

It is a good example of minimal-footprint integration: own one field of one object, under a supported exemption, and let the platform keep the rest.

A Certificate in the Wrong Namespace

This bug is worth telling in full, because the failure was silent in an instructive way.

TLS certificates for the gateway were originally declared in `aks-istio-ingress`, the namespace where the gateway lives. Reasonable. Flux applied them successfully. They never issued.

The cause: `aks-istio-ingress` is a managed namespace protected by admission policy. Flux is exempt from that policy, so it could create the Certificate object. **cert-manager is not exempt**, so it could not write status back to it. Without a status update there was no certificate request, no ACME order, and no certificate, forever.

And nothing reported an error. Flux saw an object it had applied successfully and called it healthy. The smoke test at the time checked that pods and Services existed, which they did. A status-less Certificate looks exactly like a Certificate that has not finished yet.

The fix was to issue certificates into `platform`, where cert-manager already works, and then bridge back: a **ReferenceGrant** in `platform` explicitly permits the Gateway in the other namespace to read the secret, and the listener names `platform` in its certificate reference. Issuance now completes in roughly thirty seconds instead of hanging indefinitely.

Two changes to the process followed, and they matter more than the fix. The gateway group became a real gate on certificate issuance rather than reporting success regardless. And **the smoke test now performs an actual HTTPS handshake**, because the previous test could not distinguish a working TLS configuration from one that had never issued a certificate at all.

The lasting cost is cognitive: the trust path now spans two namespaces, and a reader has to follow a ReferenceGrant to understand why a listener resolves.

Port 80 Is Always Open

The most important operational caveat in the ingress design, stated bluntly by the documentation and worth repeating here.

HTTP on port 80 serves the API routes in every mode, including the TLS modes.

The reason is mechanical rather than careless. The base Gateway keeps an HTTP listener because Let's Encrypt's HTTP-01 challenge requires one to validate domain ownership. The OSDU route definitions do not name a specific listener, so they attach to every compatible listener, HTTP included. And nothing redirects HTTP to HTTPS.

The consequence: a client that uses `http://` will send its bearer token **in plaintext**, even on an environment that has a perfectly good certificate sitting right there. The security of the connection depends on the client choosing correctly, and the server does not enforce it.

In `ip` mode there is no HTTPS at all, which is why the documentation restricts it to isolated debugging and says so repeatedly.

This is a good general lesson about managed platforms: “there is a certificate” and “the connection is encrypted” are different claims.

Six Places an Endpoint Can Fail

Because the request path crosses several controllers in several namespaces, the ingress guide organizes troubleshooting by **boundary** rather than by symptom, which is a genuinely better diagnostic model.

Symptom	Boundary	Owner
DNS lookup fails	Name resolution	Azure DNS, or ExternalDNS in <code>dns</code> mode
Connection refused	Load balancer	AKS ingress Service
TLS handshake fails	Certificate	cert-manager
Connected but no route matches	Route acceptance	Gateway and HTTPRoute
503 from the gateway	Backend availability	Service, endpoints, pod readiness
404 from the application	Application routing	The OSDU service itself

The sharpest distinction is at the bottom: **a 404 and a 503 are not interchangeable**. A request that returns 404 has already resolved DNS, completed a TLS handshake, matched a route, and reached a healthy application that simply does not recognize the path. A 503 means it never got there. These are different problems in different subsystems, and treating them as “the API is broken” wastes the information the status code already gave you.

One more trap: **a route existing in Kubernetes does not mean the Gateway accepted it**. Route objects carry status conditions reporting whether their parent Gateway took them. An unaccepted route is present, well-formed, and completely inert.

Two Documented Rough Edges

The project flags both rather than leaving them to be discovered.

Switching ingress modes is not a migration. It is done by re-running `spi up` with the new mode and the original location, partitions, repository, and branch. This re-runs infrastructure provisioning, rewrites the ingress configuration, and repoints Flux. There is no zero-downtime path, and because Azure’s incremental deployment model does not delete resources that a template stopped mentioning, leaving `dns` mode means old DNS records and identity assignments need to be inspected by hand.

The `--dns-zone` flag is incomplete. Bicep needs both the zone’s name and its resource group. Passing `--dns-zone` sets the name and *skips the discovery step that would have found the resource group*, and there is no flag to supply it. Automatic discovery, which works when the subscription contains exactly one zone, is currently the only complete path. Naming the zone explicitly is worse than saying nothing.

Part VII: Making an Empty OSDU Useful

A freshly deployed OSDU is running and cannot do anything.

Every service is healthy. Every probe passes. And a request to store a record fails, because three pieces of state that no service creates for itself do not exist yet, and the schema database is empty.

The Three Things That Must Exist

A partition record. OSDU is multi-tenant, and a partition is a tenant. Every service, on every request, asks the partition service which database, which message bus, and which storage account belong to this tenant. With no partition record, no service can resolve its own backend.

Entitlements root groups. Every authorization check goes to the entitlements service. With no groups, every check fails. Not “denies the request” but “has nothing to evaluate.”

A default legal tag. OSDU enforces compliance metadata on ingestion. Every record must reference a legal tag. With none defined, there is nothing valid to reference and ingestion cannot proceed.

Bootstrap as Data

The obvious implementation is a script in the CLI that waits for services to be ready and calls their APIs. The project rejected this for the same reason it rejected imperative deployment generally: it puts essential state creation outside Git, makes it invisible, and means a fresh environment is only correct if someone remembered to run the second command.

Instead, bootstrap is a **Helm chart** that renders one-shot Kubernetes Jobs, reconciled by Flux like everything else. Four jobs per partition: partition initialization, entitlements initialization, legal tag seeding, and member seeding. They run under the same Workload Identity ServiceAccount as the services, obtain tokens the same way, and are placed in the dependency graph after the core services are healthy.

Multi-partition support falls out for free. Deploying with two partitions renders two sets of jobs. It is a list argument, not a code path.

The failure policies differ deliberately, as noted earlier: legal seeding is visible but non-gating, while failed member seeding blocks deployability, because the root groups it creates are what make positive tests possible at all.

Loading 1,386 Schemas

The schema service comes up healthy with an empty database. Any request naming a record type fails with `schema not found`. OSDU’s shared schemas number about 1,386, and loading them is mechanical, idempotent, and needs to happen exactly once per environment.

The same reasoning applies and the same answer follows: a Flux-managed Job, running the community schema-load image, wired into the dependency graph after initialization. Its bootstrap script treats “already exists” as success, so re-running it is harmless. Re-running it is an explicit act: delete the Job and reconcile.

The elegance is that **the cluster itself now holds the “has this been loaded” state**, in the form of a completed Job, rather than that knowledge living in someone’s memory or a runbook.

A dependency is disclosed: the loader image comes from the community registry, and the project notes that mirroring it to its own registry is a plausible follow-up if retention becomes a problem.

The Image Lock

Which version of each OSDU service is running? This turns out to be a surprisingly deep question, and the answer is one of the more transferable ideas in the project.

The community registry publishes images under mutable tag patterns and a `latest` tag. Neither is stable. Tags get pruned. `latest` moves.

Two obvious approaches both fail:

Pin the tag in each service's deployment configuration. Now every image bump is a commit touching a dozen files, chart edits and image bumps are coupled, and the diffs are noise.

Follow `latest`. Now the environment changes under you mid-test, which is the exact hazard that GitOps suspension exists to prevent.

SPI Stack introduces a third option: a **single ConfigMap**, `osdu-image-lock`, holding the resolved image, tag, and digest for all fourteen images including the schema loader. Deployment manifests reference variables; Flux substitutes the values from that ConfigMap at apply time.

The ConfigMap is a lock file, in the same sense as a package manager's lock file, and it behaves like one:

- A deployment resolves images **once** and records exactly what it resolved.
- Re-running deployment **preserves** the existing lock. Images do not move because you re-ran a command.
- Moving forward requires asking: `--refresh-images`.
- Because digests are recorded, the environment is reproducible rather than merely re-derivable.

Everything in Part VIII builds on this one object. Once the running image set is a single piece of data that Flux watches, **changing what is deployed becomes an edit to that data**, which is a much easier thing to authorize, audit, and undo than a cluster mutation.

Part VIII: The Shared Environment

Everything so far describes deploying an OSDU environment. This part is about something different and less obvious: **SPI Stack is also the test target for eight other repositories**, and a surprising amount of its design exists to serve that role.

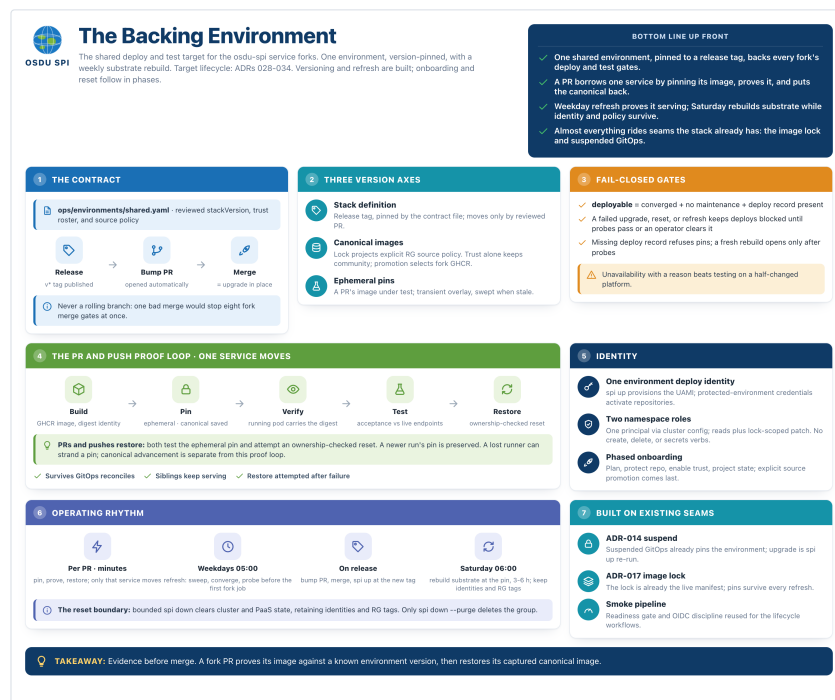
The Setup

Azure maintains forks of the OSDU services, one repository per service. Each builds container images. Each needs to answer, before merging a pull request, a question that cannot be answered by unit tests: *does this build actually work inside a real OSDU deployment, talking to real Cosmos DB, real Service Bus, and the other nine services?*

Standing up a full environment per pull request is not viable. It takes the better part of an hour and costs real money.

So there is **one standing shared environment**, and all eight repositories deploy into it, test against it, and get out.

This creates a set of problems that are really about trust and coordination between repositories that do not trust each other by default.



The shared environment has four distinct lifetimes, advanced by different verbs: identity, substrate, instance, and version contract.

Deliberately Slower Than a Branch

The first question: which version of SPI Stack does the shared environment run?

Tracking the main branch is the obvious answer and the wrong one. Every merge to SPI Stack would change the test target underneath eight repositories' continuous integration. A platform regression would break all eight simultaneously, and none of them would have any reason to suspect that the cause was a change in a repository they do not watch.

So the shared environment is pinned to a **release tag**, never a branch, and its entire definition lives in one reviewed file, `ops/environments/shared.yaml`, which declares the environment name, the stack version, the profile, the location, the ingress mode, and the trust roster. Moving the environment forward means merging a pull request against that file. A release automatically opens the bump pull request.

The consequence is accepted rather than mitigated: **freshness now costs a full release-and-bump cycle, and there is deliberately no faster path**. A hotfix route shorter than that does not exist, because a hotfix route shorter than that is exactly the thing that would let an unreviewed change reach eight repositories' test gate.

Pinning to a commit hash was rejected as less legible; a separate environment repository was rejected as splitting the reviewed definition from the code it declares. One operational requirement follows: **release tags must be protected**, because a movable tag would make the same pin name mean different code over time, which quietly destroys the entire property.

Four Lifetimes, Four Verbs

The environment is not simply “up” or “down.” It has four layers with genuinely different lifetimes, and conflating them is how people lose data or break things.

Layer	Contains	Changed by
Identity	Resource group, managed identities, federated credentials, naming and policy tags	Onboarding; only a purge destroys it
Substrate	The Kubernetes cluster, Azure PaaS, the Flux extension	A full reset
Instance	Workloads, the image lock, middleware data	Refresh, upgrade, and fork deploys
Version contract	The reviewed declaration file	A merged pull request

Five verbs act on them, each with defined semantics so that nobody improvises during an incident:

- **status**: report health and whether the environment will accept a deploy.
- **refresh**: prove the environment is still serving. Runs on a weekday schedule.
- **upgrade**: re-run deployment at a new stack version.
- **reset**: full teardown and cold rebuild at the pinned version. Weekly.
- **teardown**: destroy the environment, including identities.

A middle-tier reset that rebuilt only the workloads was explicitly rejected. The reasoning is sharp: **accumulated state lives in Azure PaaS, not just in the cluster**, and a reset that only rebuilds the cluster leaves Cosmos records and entitlements groups behind, which means it does not do what its name promises.

There is a second argument for the full weekly rebuild that generalizes well beyond this project: **the rebuild path is exercised every week, so it is continuously proven to work**. A recovery procedure that is only attempted during an actual incident is not a recovery procedure. It is a hope.

Failing Closed

The environment carries a **maintenance flag**, and its semantics are the interesting part.

When a lifecycle workflow is about to change the environment, it sets the flag. Fork deploy jobs check it and refuse to proceed. When the workflow finishes and has verified that the environment is genuinely healthy, meaning Flux has converged and gateway probes pass, it clears the flag.

If the workflow **fails**, the flag stays set.

Clearing it on the failure path was explicitly rejected. The result is that a broken 5 a.m. refresh leaves the environment refusing deploys until a human looks at it. Eight repositories' pipelines block, visibly, with a named reason.

That is the intended behavior. The alternative is eight repositories deploying into a sick environment and getting failures that appear to be their own, producing eight investigations of a problem none of them caused.

Correctness over partial availability, chosen deliberately and with the cost understood.

Readiness as an API

Fork pipelines need to ask two questions before deploying: *is this environment healthy enough to accept a deploy*, and *what is it actually running?*

They could inspect the cluster directly with `kubectl`. The project rejected this specifically because it would couple eight repositories to the internal names and labels of SPI Stack's Kubernetes objects, freezing them as a public interface by accident.

Instead, `spi status --json` emits a **versioned envelope** with a stable schema, a declared API version, and typed exit codes. Exit 0 means deployable. Exit 2 means blocked and names the reason.

One field separation carries most of the design's weight. `ready` and `deployable` are **not** the same field:

- `ready` means Flux has converged and the workloads are healthy.
- `deployable` means it is acceptable to deploy right now.

An environment can be perfectly healthy and not deployable, because maintenance is in progress. Folding maintenance into readiness was rejected; they are genuinely different questions and a consumer needs to be able to tell them apart.

Alongside this, `spi up` writes a **deploy record**, stored both as resource group tags and as a ConfigMap, capturing the verified commit, the CLI version, and the maintenance state. The same internal collector feeds the human-readable tables and the JSON, specifically so the two surfaces cannot drift apart and disagree.

Two honest caveats are recorded. The JSON envelope is now a compatibility contract, so renaming a field is a breaking change. And environments provisioned before the deploy record existed have none, so they fail closed and report themselves undeployable until a fresh deployment writes one.

A Deploy Is a Lock File Edit

Now the central mechanism, and it is a genuinely elegant piece of design.

A fork has built a container image. It is identified by a **digest**, a content hash, not a tag. It needs to run in the shared environment for the duration of a test, and then the environment must go back to how it was, without disturbing the other nine services.

The direct approach is to run the Kubernetes command that updates the deployment's image. **This does not work**, and the reason is a nice demonstration of what GitOps actually means: Flux is continuously reconciling the

deployment against the manifests. It will notice the change and revert it. The cluster is not the source of truth, so writing to the cluster is not a deploy.

So the deploy writes to the thing that *is* a source of truth: the **image lock** from Part VII. The CLI writes a pin into that ConfigMap, Flux observes the change, and rolls out the new image through the normal path.

The lock edit is the deploy. Flux does the rest.

Other approaches were considered and rejected: publishing a per-deploy Helm chart and repointing the release (heavy, and creates artifacts to garbage collect); a cluster-wide deploy lock (serializes all eight repositories against each other); suspending the owning reconciliation group for the pin's duration (blinds Flux exactly when you most want it watching).

Ownership, Which Is the Hard Part

Concurrency is the real problem. Eight repositories, many pull requests, and a shared mutable object. Pins must be cleaned up, but cleanup must not delete somebody else's pin.

The pins carry **provenance**, stored as a JSON annotation on the ConfigMap. Each pin records the image and digest, the source repository, the source commit, the owning workflow run identifier, a flag marking it ephemeral, a timestamp, and, critically, **the canonical image it replaced**, captured at the moment of the overwrite.

Every operation checks ownership:

- Restore uses `--if-run`, which removes the pin **only if this run still owns the slot**. If a later run has since pinned the same service, the older run's cleanup correctly does nothing.
- Verification confirms not just that the rollout completed but that a running pod's actual image digest matches what was pinned.
- The restore target is the canonical image captured with the pin, not whatever is canonical now.

Two failure modes are anticipated rather than discovered:

A stranded pin. A run is cancelled, or its token expires, or the runner disappears. The cleanup never happens and a pin sits there. A scheduled sweep handles it, removing only pins marked ephemeral whose owning run has reached a terminal state, or, if the run's status cannot be determined, only pins older than a threshold longer than any plausible deploy-and-test budget. The threshold exists so the sweep can never race a deploy that is still running.

A pin replaced mid-flight. If a newer run pins the same service, the older run's verification step fails and names the replacement. The recovery model is not delete-and-retry. It is **verify, find out who owns the slot now, and act on that**.

Two Limits Stated Honestly

The permission boundary is the lock object, not the individual service entry. **Any trusted repository can technically rewrite any other service's lock entry.** The project accepts this for an environment with a single trust level and notes that separate trust levels would require separate writable locks or separate environments.

And a candidate image that passes rollout verification can still be behaviorally broken. Verification proves the right bits are running, not that they are correct.

Trusting a Repository Without Trusting Its Pull Requests

How does a fork's CI job authenticate to Azure at all?

GitHub Actions can obtain a short-lived OIDC token asserting facts about the running job, and Entra ID can be configured to trust specific assertions. The natural configuration trusts the subject `repo:org/fork:pull_request`.

That is a security hole. **The `pull_request` subject is minted for any pull request job, including one from an untrusted fork opened by a stranger.** Using it to anchor a privileged credential grants that credential to anyone who can open a pull request.

The solution uses a GitHub feature designed for this: a **protected deployment environment**. The federated credential names `repo:<org>/<fork>:environment:spi-stack`, which is only ever minted for a job that GitHub has admitted into that protected environment, subject to whatever approval rules it carries.

Trust is expressed as a GitHub environment credential, not as a list of repository names hidden in code.

The command `spi onboard` activates a repository in phases: it configures the repository's protected environment and stamps the connection values it needs; it adds the federated credential to the deploy identity and to the member and no-access test identities; and it projects the resulting trust roster into the image lock so that pin operations can validate claimed provenance. It plans read-only by default and only writes when asked, and it is explicitly **resumable rather than transactional**, so a failure partway through does not unwind trust that was already established.

The permissions granted are deliberately narrow: two namespace-scoped roles, one allowing writes to the image lock specifically by name, one allowing reads of workload state. No create verb, no delete verb, and **no access to Kubernetes Secrets**.

Trusted and Canonical Are Different Questions

A subtle distinction, and the project is right to separate it.

For each service there are two plausible sources of the “real” image: the OSDU community registry, or the Azure fork's own published image. These are different questions from trust:

- *Is this repository allowed to deploy here?* Recorded by the federated credential.
- *Is this repository's image what the environment runs by default?* Recorded separately, as a resource group tag per service.

Inferring the second from the first was explicitly rejected, and the reason is sound: **onboarding a repository so it can test should not silently promote its images to canonical**. Those are different decisions made by different people at different times.

Because they are separate, services can be migrated one at a time. Some run community images while others run fork images, during a transition that can take as long as it needs. Flipping the whole fleet at once was rejected.

One inherited risk is disclosed: pointing at fork-published images couples the environment to that registry's retention policy. A refresh that stalls long enough could leave a canonical image aging toward a digest that no longer exists.

Why the Client ID Must Survive

The last decision is small, concrete, and a good illustration of designing around a real constraint.

Each trusted repository stores five values to connect to the environment. Four are stable. The fifth is the deploy identity's **client ID**, which is minted when the managed identity is created.

Azure managed identities have **no soft delete**. Delete one and recreate it, and the new one has a different client ID, permanently.

Now recall that the shared environment is fully rebuilt every week. A naive teardown-and-rebuild would mint a new client ID every Saturday and strand all eight repositories, each requiring a manual update to a stored secret. Weekly.

So `spi down` was changed to **delete resources individually while leaving the managed identities, the resource group, and its tags intact**. A rebuild never rotates the client ID. Nobody has to touch a repository after a reset.

The retained tags carry more than the identity: the naming suffix, so regenerated resources get the same globally unique names, and the per-service canonical source policy, so the rebuild restores the same image lineage.

Full destruction is a separate, explicit path: `spi down --purge`, which first inventories and removes role assignments the identities hold **outside** the resource group, such as a grant on a DNS zone in another group, confirms that cleanup succeeded, and only then deletes everything.

The costs are named: ordinary teardown is slower and not atomic, so a failure can leave partial state, though re-running is safe. And purge needs permission to inspect and delete grants at scopes beyond the resource group, which is a strictly larger permission than deleting a resource group.

Part IX: Fifty Minutes, Start to Finish

A walkthrough of what actually happens when someone runs `spi up --env dev1`, which ties together everything above.

Before Anything Is Created

Preflight. The CLI queries the target subscription’s catalogue to resolve which availability zones are usable for the system node pool’s machine size in this region. If the subscription cannot use the full set, it stops here, before creating a single resource, with an error naming the size and region.

Resource group. Verify the Azure login. Create or reuse the group. Persist the naming suffix as a tag, so that globally-unique names stay stable across future runs.

Azure

The cluster, roughly thirty minutes. Deploy the AKS template. This is the bulk of the wall clock and there is nothing to be done about it. Then fetch kubeconfig, enable Istio CNI chaining with the imperative call the declarative path cannot express, and grant the deploying user cluster-admin access.

Platform services. If a soft-deleted Key Vault with a matching name exists from a previous environment, recover it. Then deploy the main template: identities, federated credentials bound to the cluster’s OIDC issuer, Key Vault, container registry, the Gremlin graph, shared storage, and for each partition a Cosmos DB account, a Service Bus namespace, and a storage account.

The Cluster

Bootstrap. The CLI writes the things Flux will need but cannot create for itself: namespaces, the credential seed and the middleware Secrets derived from it, storage classes, Gateway API definitions, the annotated ServiceAccounts that make Workload Identity work, several configuration ConfigMaps including the detected Istio revision, the trusted-repository projection, and the three Istio identity policies from Part V.

Flux activation, ten to fifteen minutes. Deploy the Flux template with the repository, the branch or tag, the profile, and the ingress path. **Flux begins reconciling immediately**, concurrently with the CLI’s remaining work. There is no clean handoff point where one stops and the other starts.

Runtime secrets. Write the middleware passwords and derived endpoints into Key Vault. Notably, this does **not** wait for Elasticsearch or Redis to be ready, because the passwords were generated during bootstrap and are already known. There is nothing to wait for.

The Pause

Source finalization. Wait up to ten minutes for the Git source to appear. Resume it, reconcile it, and **verify that the artifact revision matches what was requested**. A missing, unready, or wrong revision fails the deployment rather than proceeding with something unexpected. Then suspend the source and write the deploy record.

The CLI exits successfully.

Meanwhile, Flux Is Still Working

The dependency graph is running: node pools, operators, middleware, the bootstrap layer, OSDU services, partition and entitlements initialization, then schema loading, legal seeding, and the reference services.

Schema loading alone carries a **150-minute deadline**. On a `core` profile the tail can be long.

Five Milestones, Not One

The documentation insists on distinguishing these, and an operator who conflates them will misdiagnose:

1. The CLI exited successfully.
2. The Git source has an artifact.
3. All workload groups report ready.
4. The initialization Jobs have completed.
5. **An authenticated API request succeeds.**

Only the fifth means the environment works. Prior smoke-test observations put fresh provisioning at roughly forty-five to fifty minutes, and the project is careful to frame that as an observation from a specific region and release, not a guarantee.

Two smaller traps are called out: a pod in the `Running` phase is not necessarily *ready*, and a Job that has completed correctly is not supposed to still be running.

Coming Back Down

`spi down` deletes resources individually and waits up to forty-five minutes. Networking comes apart in dependency order: subnet associations, then NAT gateway, then public IP, then the virtual network.

Managed identities, the resource group, and its tags survive, for the reasons in Part VIII. Success means a fresh inventory shows only the identities remaining and the AKS-managed node group is gone.

Kubeconfig cleanup happens only after deletion is confirmed, and it checks the recorded API server address first, so that a same-named cluster in a different subscription does not lose its context.

`--purge` additionally removes the identities, but only after discovering and successfully removing every role assignment they hold outside the group. If any of that fails, the purge fails rather than orphaning a grant.

Part X: The Things That Are Not True

A list worth having, because each entry is a plausible assumption that this system's documentation explicitly contradicts. Collectively they are the best evidence of the project's intellectual honesty.

"The command finished, so it is ready." No. Flux is still converging. Readiness is a later milestone that the CLI does not wait for.

"GitOps is suspended, so nothing changes." No. Suspension stops *fetching new commits*. The cached revision keeps being applied, live edits keep being reverted, external chart repositories keep their own schedules, and other controllers keep running.

"Async indexing works." Not with the default community images. The `indexer-queue` service builds a Service Bus connection string regardless of Workload Identity and reads the `DISABLED` placeholder.

"There is a certificate, so the connection is encrypted." Not necessarily. Port 80 serves the API routes in every mode, nothing redirects, and a client using `http://` sends its bearer token in plaintext.

"The dry run showed me what will happen." Partially. It previews two of three templates, cannot produce the cluster's OIDC issuer and therefore omits every federated credential, skips DNS discovery, and still creates the resource group.

"Deleting the Secret rotates the password." No. The chart consumes the CLI-created Secret and the CLI reuses the persistent seed. You get the same password back. There is no rotation command at all.

"Refreshing the CA Secret rotates trust." No. The Java truststore is built by an init container at pod startup. Running pods keep the old truststore until they are recreated.

"The role assignment is missing." Maybe not. Cosmos DB data-plane grants are Cosmos-native assignments that do not appear in standard Azure role assignment queries, and propagation can lag five to fifteen minutes.

"The teardown job was green, so it was deleted." No. The CI workflow requests deletion asynchronously and tolerates failure. Green means the request was made. A separate sweeper exists precisely because that is not enough.

"The route exists, so traffic flows." Not necessarily. A route object can be present, well-formed, and not accepted by its parent Gateway.

"Re-applying the manifest will retry it." Not for a Helm release that has exhausted its retries. The controller only reacts to a generation change, and identical content does not change the generation.

"The smoke test passed, so the API works." The scheduled smoke test runs the `bare` profile, which deploys no OSDU services at all. It proves infrastructure and GitOps readiness. Even a full run only proves the network and TLS path, not authenticated API behavior.

"Profiles save money." No. All profiles provision the full Azure estate. They differ only in what runs on top.

Part XI: The Register

One last thing, because it is unusual enough to be worth naming.

Most projects that keep architectural decision records keep them as a log: dated entries with a status field, superseded records left in place with pointers to their replacements, an accumulating archaeological record.

SPI Stack keeps a **register** instead, and the distinction is deliberate.

Each record states the ruling **as it stands today**. There are no dates, no status fields, no deciders, no amendment trails. A decision that changes is **rewritten in place** through a pull request, with the old choice preserved only as a one-line rejected alternative when the contrast still teaches something. A decision that stops standing alone is folded into whichever record owns its subject, and the corpus is renumbered to stay contiguous.

The argument is that **Git history already carries chronology, authorship, and every prior form of every record, far better than a status field does**. Duplicating it in the document makes the reader work out which parts are current. The register answers “what is true now”; `git log` answers “how did we get here.”

There is a stated test for whether a record deserves to exist at all, and it is operational rather than aesthetic: **without this record, would a competent contributor plausibly re-propose the rejected alternative, or walk into the trap it documents?** If not, the content belongs in a design document or in the code. That test is why the register has thirty-four entries rather than two hundred.

The house style follows from it. Rejected options get exactly one line each, preserving their genuine advantage without re-litigating the argument. There are no acceptance checklists, because those belong in the pull request that implemented the decision. There is no incident narrative, because a specific outage ages badly while the structural problem it revealed does not. Consequences mix good and bad, unsorted, **and the honest limitation is supposed to lead**.

That last convention explains the character of the entire corpus. The documentation says that all services share one Azure identity and that this does not isolate them. It says the indexer path does not work. It says there is no rotation. It says port 80 is open. Not in footnotes, but in the position where most documentation would put a benefit.

It is worth considering how much of this document’s usefulness comes from that one stylistic rule.

Glossary

ADR (Architecture Decision Record) A short document recording one significant design decision, its context, the alternatives rejected, and the consequences accepted.

AKS Automatic An Azure Kubernetes Service tier where Microsoft manages node provisioning, service mesh, admission policy, and monitoring, in exchange for accepting opinionated defaults.

Bicep Azure's declarative infrastructure language, used here to describe all Azure resources.

Canonical source The repository treated as the default origin for a service's image, recorded separately from whether that repository is trusted to deploy.

Credential seed A Kubernetes Secret holding generated middleware passwords, created once and reused on every subsequent deployment.

Deploy identity A managed identity per environment, used by CI jobs to authenticate. Its client ID survives teardown so trusted repositories do not need reconfiguring.

Deploy record Machine-readable state recording the verified commit, CLI version, and maintenance status of an environment.

Deployment Safeguards AKS Automatic's non-bypassable admission policy requiring security context, resource limits, and probes on every pod.

Digest A content hash identifying a container image exactly, unlike a tag, which can be moved.

Entitlements The OSDU service that answers authorization questions; backed here by a Cosmos DB Gremlin graph.

Ephemeral pin An image pin written by a CI run, marked for automatic cleanup and owned by that run.

Federated credential Configuration in Entra ID trusting an external OIDC issuer's token for a specific subject, removing the need for a stored secret.

Flux CD The GitOps engine that reconciles Kubernetes workloads from Git; installed here as an Azure-managed cluster extension.

Image lock A ConfigMap recording the resolved image, tag, and digest for every OSDU service; the single source of truth for what version runs.

Ingress mode One of three self-contained exposure configurations: `azure`, `dns`, or `ip`.

Inventory owner The one Flux group responsible for applying and pruning a given Kubernetes object. Exactly one per object.

Karpenter A node provisioner that creates virtual machines on demand to fit pending pods.

Maintenance flag Environment state that blocks new deploys during lifecycle operations. Fails closed: it stays set if the operation fails.

OSDU The Open Subsurface Data Universe: an open standard and reference implementation for energy industry data.

Partition An OSDU tenant. Each gets its own Cosmos DB account, Service Bus namespace, and storage account.

Profile The workload scope of a deployment: `bare`, `minimal`, or `core`.

ReferenceGrant A Kubernetes resource explicitly permitting a cross-namespace reference, used here so a Gateway can read a certificate from another namespace.

Service pin An override in the image lock directing one service to a specific image.

Service provider interface OSDU's pluggability seam separating shared business logic from cloud-specific implementations.

Suspension Stopping Flux from fetching new Git commits. Not a freeze: reconciliation of the cached revision continues.

trust-manager A controller that distributes certificate authorities across namespaces declaratively.

Workload Identity The mechanism by which a pod exchanges a signed Kubernetes token for an Azure access token, eliminating stored credentials.

Appendix: The Decision Register

The thirty-four standing decisions, each stated as its ruling.

#	Title	Ruling
001	Azure PaaS for Data	Every data service with a managed equivalent runs as Azure PaaS; the stack is Azure-only by design.
002	AKS Automatic	AKS Automatic with a Kubernetes 1.36 floor; Karpenter, managed Istio, and Safeguards come from the platform.
003	In-Cluster Middleware Scope	Exactly three stateful systems stay in-cluster: Elasticsearch, Redis, and PostgreSQL for Airflow.
004	Local Helm Chart	One local chart bakes Safeguards compliance into its templates; releases override only image, env, and resources.
005	Workload Identity	One shared managed identity federated with a ServiceAccount carries all OSDU PaaS access. No stored credentials.
006	Three-Namespace Model	Workloads split across <code>foundation</code> , <code>platform</code> , and <code>osdu</code> .
007	Layered Ordering	The stack reconciles as ordered layers wired by explicit dependencies; same-layer work runs in parallel.
008	Bicep for Provisioning	Azure resources are hand-written Bicep at pinned API versions; the CLI owns only the lifecycle seams.
009	Flux CD Extension	The AKS native Flux extension reconciles all in-cluster state from one configuration.
010	Key Vault + ConfigMap	Secrets split by class: Entra tokens, Key Vault values, and CLI-generated Kubernetes Secrets.
011	trust-manager CA Distribution	Bundles mirror middleware CAs into <code>osdu</code> ; a DestinationRule disables mesh mTLS toward Redis.
012	Three Ingress Profiles	Ingress is one of three self-contained trees selected by flag; <code>azure</code> is the default.
013	Schema Load Job	A Flux-managed Job runs the community schema loader under Workload Identity.
014	Suspend After Deploy	Deployment ends by suspending the Git source; the environment stays pinned to its deploy commit.

#	Title	Ruling
015	Bootstrap Helm Chart	A chart renders per-partition one-shot Jobs creating the partition, entitlements, and legal tag.
016	Istio JWT Projection	Request authentication plus a Lua filter validate Entra tokens and project the headers the provider expects.
017	Image Lock	Images resolve into one ConfigMap and Flux substitutes them; refreshes are explicit and preserve pins.
018	NodePools as Manifests	Node pools are Flux-managed manifests: a fixed on-demand platform pool and a spot-capable OSDU pool.
019	osdu-flux Namespace	The CLI creates a dedicated namespace and seeds all SPI-owned GitOps objects there.
020	Opt-In App Insights	Telemetry deploys only when requested; the default is off, and the agent is kept inert.
021	minimal Profile	Profile values are <code>bare</code> , <code>minimal</code> , and <code>core</code> ; the unbacked <code>full</code> value is removed.
022	Certificates in platform	Certificates issue into <code>platform</code> ; ReferenceGrants let the managed-namespaces Gateway read them.
023	Entra-Only Data Plane	Local auth is disabled on every Cosmos account and Service Bus namespace.
024	Windows Batch Shim	Batch shims launch through one escaped command line at a single audited chokepoint.
025	Single Inventory Owner	Every object has exactly one Flux owner; all ingress modes render the Gateway under one name.
026	Bind Managed Ingress	Ingress binds to the AKS-provisioned ingress Service; the DNS label is applied via Flux.
027	Resolved Availability Zones	The CLI resolves usable zones from the subscription before deploying; a restricted zone is a preflight error.
028	Version-Pinned Environment	The shared environment deploys a release tag, never a branch; its declaration is one reviewed file.

#	Title	Ruling
029	Lifecycle Verbs	Refresh changes runtime state, upgrade re-runs at a new tag, and only a full reset clears both layers.
030	Machine-Readable Status	Status emits a versioned envelope gating on <code>deployable</code> , with <code>ready</code> as a separate field.
031	Ephemeral Lock Pins	A fork deploy is a digest pin on the image lock, marked ephemeral and owned by its workflow run.
032	Deploy Identity and RBAC	One deploy identity per environment; a repository is trusted by federating its protected environment.
033	Canonical Source Policy	Credentials record trust; tags record canonical source policy. They are separate decisions.
034	Identities Survive Teardown	Teardown keeps managed identities, the group, and its tags; only purge removes them.

Sources

Everything in this document derives from the `Azure/osdu-spi-stack` repository:

- `README.md` and `docs/architecture.md`
- `docs/decisions/` , the thirty-four-record decision register
- `docs/design/` , the nine subsystem guides: deployment lifecycle, Bicep architecture, Flux reconciliation, Workload Identity, gateway and ingress, secret lifecycle, environment lifecycle, fork deployment, and CI smoke
- `docs/diagrams/` , the source of the figures reproduced here

The repository is licensed under Apache 2.0.